

Join at menti.com | use code 4607 8811



Ali so naslednje trditve pravilne

v eni datoteki lahko imam izvorno kodo več razredov

izvorno kodo prevedemo z javac lmeDatoteke.class

v eni datoteki lahko imam več public razredov

privzeta vidnost metod je private

konstruktorji so metode

konstruktorjev je lahko več

vsi objekti zagotavljajo vsaj metodo toString()

izvajanje lahko začnemo s poljubnim razredom

Strongly disagree

Strongly agree

1

OO koncepti

- **objekt** ("object")
- **ograjevanje** ("encapsulation")
- **razred** ("class")
- vmesnik ("interface")
- **dedovanje** ("inheritance")
- delegiranje ("delegation")
- komunikacija s sporočili
- polimorfizem ("polymorphism")

2

Prejšnjič

enum

naštevni tipi – določimo nabor vrednosti, ki jih lahko spremenljivka tega tipa zavzame, uporaba: `ObdobjeVezave.MESECNA`
tudi za naštevni tip (enum) svoja datoteka z vmesno kodo

dedovanje - **extends**

- proženje ustreznega konstruktorja nadrazreda s `super(...)`
- s `super` pa si lahko pomagamo tudi v primeru, ko želimo izkoristiti metodo nadrazreda, ki smo jo sicer v podrazredu predefinirali ("override-ali,,")

java.lang.Object

- vsi dedujejo od tega razreda
- spremenljivka tega tipa lahko predstavlja poljuben objekt, tj objekt poljubnega razreda
a pozor – na voljo se le metode, deklarirane v `java.lang.Object`

3

```
public enum ObdobjeVezave {
    MESECNA, POLLETNA, LETNA
}
```

```
private ObdobjeVezave doba;
private boolean vezavaPotekla;
private LocalDate datumVezave;
private LocalDate datumPoteka;
```

```
public VarcevalniRacun(String ime, int polog, ObdobjeVezave doba ) {
```

```
    public static final ObdobjeVezave MESECNA;
    public static final ObdobjeVezave POLLETNA;
    public static final ObdobjeVezave LETNA;
    public static
    public static
    static {};
```

```
        switch (doba) {
```

```
            case MESECNA:
```

```
                datumPoteka=datumVezave;
```

```
                System.out.println(d
```

```
                break;
```

```
                :Vezave.LETNA);
```

```
            case ObdobjeVezave.POLLETNA:
```

```
                System.out.println(d
```

```
                :Vezave.MESECNA);
```

```
VarcevalniRacun letni
```

```
VarcevalniRacun zaTas
```

```
VarcevalniRacun pol = new VarcevalniRacun("Fonza POLLETNIK",10000,ObdobjeVezave.POLLETNA);
```

4

Kreiranje objektov

Konstruktorji ("constructors")

- imajo isto ime kot razred
- lahko jih je več - imajo različne parametre (signature)
- ne vračajo ničesar
operator new **vrne referenco na kreiran objekt**
- v konstruktorju se inicializira objekt tega razreda
- konstruktor brez argumentov je privzeti konstruktor, ki ga prožijo podrazredi, zato je klic **super()** odveč, saj za to avtomatično poskrbi prevajalnik, torej je klic **super (parametri)** aktualen le, če ga kličemo z argumenti,
- če ne specificiramo konstruktorja, prevajalnik avtomatično kreira konstruktor brez argumentov, ki preprosto kliče **super()**

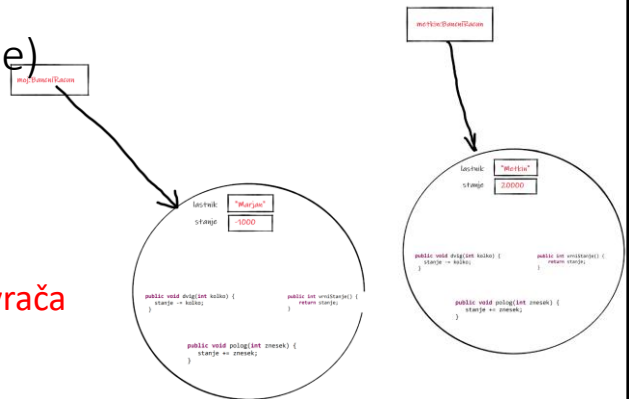
5

- instance variables - member variables
 - lahko so poljubnega tipa
 - lahko imajo privzete vrednosti
 - če privzete vrednosti ne navedemo, bodo inicializirane na:
 - 0 pri numeričnih tipih
 - false pri logičnih (boolean)
 - null pri vseh ostalih tipih

6

Metode (instančne oz. objektne)

- **instance method**
v C++ terminologiji member function
- določen mora biti **tip vrednosti, ki jo vrača**
(razen pri konstruktorjih), **sicer void**
- spremenljivke, deklarirane znotraj metod, so lokalne spremenljivke
- lokalna spremenljivka ne more prekriti druge lokalne spremenljivke ali parametra (argumenta)



7

Običajna uporaba **this**

- kakšen argument metode ima isto ime kot atribut razreda
- lokalna spremenljivka ima isto ime kot eden izmed atributov
- v enem konstruktorju je potrebno prožiti drug konstruktor

8

```

class Krog {
    double r;    // polmer

    // argument z istim imenom!
    void setPolmer (double r)
        {this.r = r;}

    // lok. sprem z istim imenom!
    void printPolmer100krat () {
        for (int r=0; r<100; r++) {
            System.out.println(this.r);
        }
    }
}

```

9

```

class Krog {
    double x,y; // koordinate
    double r;    // polmer
    // instance methods
    double obseg () {return 2*3.14*r;}
    double ploscina() {return 3.14*r*r;}

    // uporaba univerzalnega konstruktorja
    Krog (double r) { this (0,0,r);}
    Krog (Krog c) { this (c.x, c.y, c.r);}
    Krog (double xx, double yy, double rr) { x=xx;
        y=yy; r=rr;}
}

```

10

Ustvarjanje in brisanje objektov

Constructor

- ima isto ime kot razred
- lahko jih je več in imajo različne argumente
- ne vrača ničesar
- v prvi vrstici lahko kličemo `super()`
- v njem inicializiramo objekt za ta razred

Finalizer

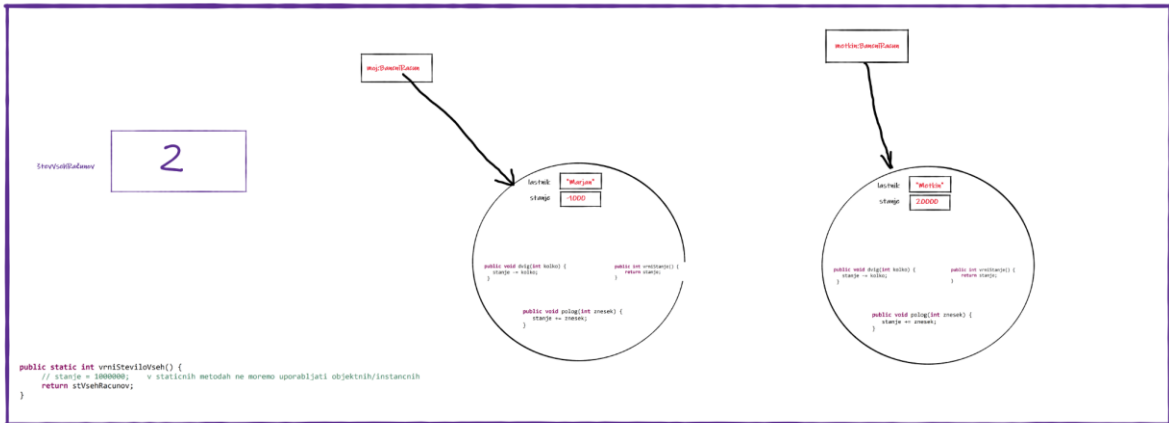
- **`finalize()`**
- izvede se tik preden zbiralec odpadkov odstrani objekt
- ni predvidljiv čas izvedbe
- z njim počistimo za objektom

11

RAZREDNI ATRIBUTI in metode

12

BančniRacun



13

Razredne spremenljivke /atributi

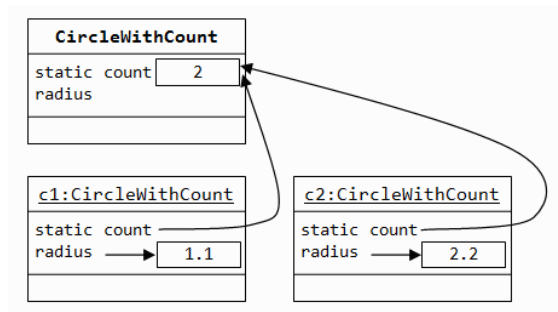
static

- skupni podatki za vse primerke (ista vrednost)
- uporabljajo/dostopajo lahko vsi primerki tega razreda
- inicializacija se izvrši samo enkrat (`static { }`)

Razredne metode

- znotraj razreda delujejo lahko le nad razrednimi spremenljivkami tega razreda
- glede metod istega razreda - prožijo lahko le razredne
- uporabimo jih sicer lahko tudi brez primerkov razreda torej preko **imena razreda, kar je dobra praksa**
- lahko pa pošiljajo sporočila drugim objektom

14



15

Dobra/slaba praksa

- do razrednih metod in spremenljivk/atributov dostopamo preko imena razreda npr.

```
BancniRacun.povecajLimit();
BancniRacun.stRacunov = 100;
```

sicer lahko tudi preko objektov, a je zavajajoče

```
BancniRacun crniFond;
```

```
...
```

```
crniFond.povecajLimit();
crniFond.stRacunov=100;
```

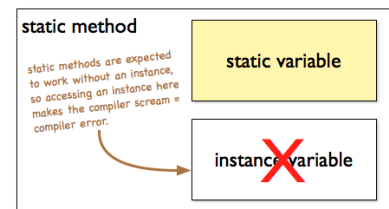
16

Razredne metode

razredne (statične) metode **ne morejo uporabljati ne-statičnih** (objektnih / instančnih) **spremenljivk in metod** (te pripadajo posameznim objektom)

Če to skušamo narediti, prevajalnik javi napako

non-static variable [or method] *Name* cannot be referenced from a static context



17

Rezervirani besedi *static* in *final*

static

- pomeni iste vrednosti za vse primerke
- pri metodah pomeni razredno metodo
- dostopamo lahko kar preko razreda in ne preko primerkov razreda (objektov)

Primer:

```
System.out.println
(" Gremo na odmor");
```

final

- omogoča deklaracijo konstant
- takšnih metod ne moremo "povoziti", so končne
- z njimi izboljšamo zmogljivost programa (prevajalnik)

Primer:

```
final float PI=3.14;
```

18

Privzete vrednosti razrednih spremenljivk

“static initializer”

```
class DelovniDnevi {
    static String[] imena;
    static int zapSt[];
    static {
        imena = new String[5];
        imena[0]="Ponedeljek";
        imena[1]="Torek"; imena[2]="Sreda";
        imena[3]="Četrtek"; imena[4]="Petek";
        for (int i=0; i<10;)
            zapSt[i]=++i;
    }
}
```

19

Prejšnjič (2)

enum

naštevalni tipi – določimo nabor vrednosti, ki jih lahko spremenljivka tega tipa zavzame,
uporaba: ObdobjeVezave.MESECNA
tudi za naštevalni tip (enum) svoja datoteka z vmesno kodo

dedovanje - extends

- proženje ustreznega konstruktorja nadrazreda s super(....)
- s super pa si lahko pomagamo tudi v primeru, ko želimo izkoristiti metodo nadrazreda, ki smo jo sicer v podrazredu predefinirali ("override-ali,,)

java.lang.Object

- vsi dedujejo od tega razreda
- spremenljivka tega tipa lahko predstavlja poljuben objekt, tj objekt poljubnega razreda
a pozor – na voljo se le metode, deklarirane v java.lang.Object

20

```

public class VarcevalniRacun extends BancniRacun {

    private ObdobjeVezave doba;
    private boolean vezavaPotekla;
    private LocalDate datumVezave;
    private LocalDate datumPoteka;

    public VarcevalniRacun(String ime, int polog,
        ObdobjeVezave doba ) {

        @Override
        public void dvig(int koliko) {

        }

        public void dvig() {

        }

        private void pripisiObresti() {}
    }
}

```

Diagram illustrating the relationship between `VarcevalniRacun` and `BancniRacun` using vertical blue double lines (||) and red arrows:

- Red arrow 1 points from `VarcevalniRacun` to `BancniRacun` constructor: `public BancniRacun(String, int)`
- Red arrow 2 points from `VarcevalniRacun` to `BancniRacun` methods: `public void dvig(int koliko)`, `public void polog(int znesek)`, `public int vrniStanje()`, and `@Override public String toString()`

21

```

public class VarcevalniRacun {
    private ObdobjeVezave doba;
    private boolean vezavaPotekla;
    private LocalDate datumVezave;
    private LocalDate datumPoteka;

    private String lastnik;
    protected int stanje;
    private static int stRacunov = 100000;

    public VarcevalniRacun(String, int, ObdobjeVezave)

    public void dvig(int koliko)

    public void dvig()

    public void polog(int znesek)

    public int vrniStanje()

    public String toString ()
}

```

Diagram illustrating the state of a `VarcevalniRacun` object named `letni`:

- Object name: `letni`
- Attributes (boxed):
 - `"Miha Letni"` (lastnik)
 - `50000` (stanje)
 - `LETNA` (doba)
 - `2026/03/04` (datumVezave)
 - `2027/03/04` (datumPoteka)
 - `false` (vezavaPotekla)
- Methods (boxed):
 - `void dvig(int koliko)`
 - `void dvig()`
 - `void polog(int znesek)`
 - `int vrniStanje()`
 - `String toString ()`

Initialization code:

```
VarcevalniRacun letni = new VarcevalniRacun("Miha Letni ", 50000, ObdobjeVezave.LETNA);
```

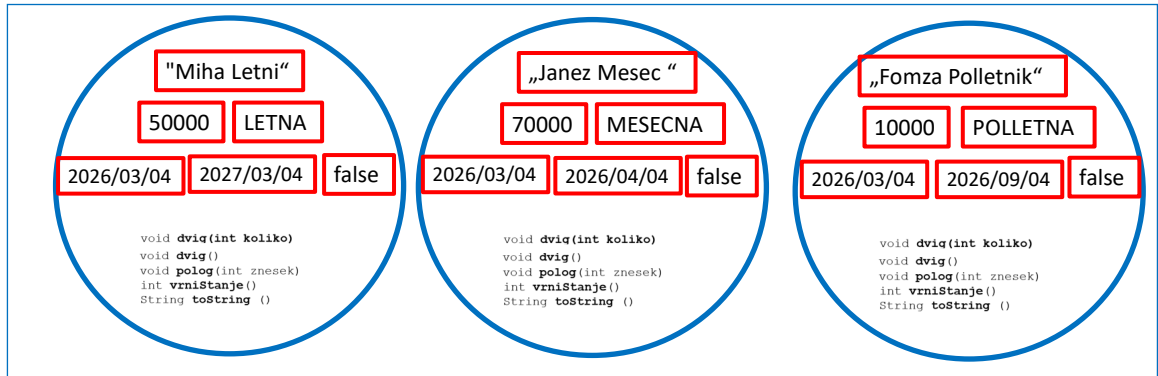
22

BancniRacun

private static int stRacunov

3

VarcevalniRacun



23

Razredne spremenljivke in dedovanje

Gre za eno in isto vrednost!

BancniRacun.stRacunov

TransakcijskiRacun.stRacunov

VarcevalniRacun.stRacunov

```
VarcevalniRacun.vrniStRacunov() ==BancniRacun.vrniStRacunov()
TransakcijskiRacun.vrniStRacunov() ==BancniRacun.vrniStRacunov()
VarcevalniRacun.vrniStRacunov() ==TransakcijskiRacun.vrniStRacunov()
```

24